# Raceline Optimisation Python Package: OptiLine

Amir Ali Farzin, Philipp Braun, and Iman Shames

Australian National University, CIICADA Lab

March, 2026


Australian
National
University

# Table of contents

# Outline

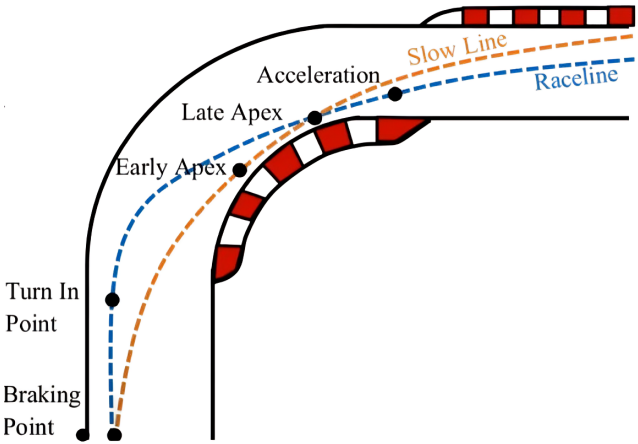
# The Raceline

**Raceline**:

- **Optimal Path** around the racetrack
- Optimality is subject to **definition**
- **Goal**: Drive **as fast as possible** around the racetrack

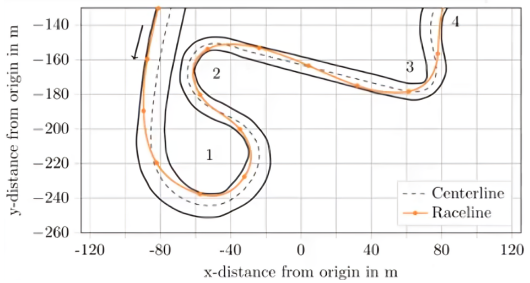
**Problem and Solution**:

- What is the fastest way to do a lap?
- Optimisation

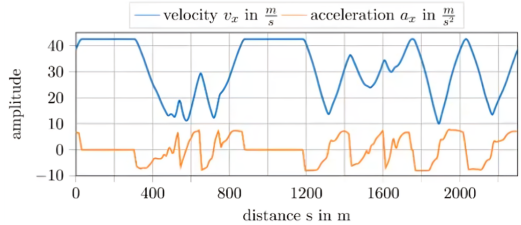


# Path and Trajectory

- **Path Planning:**
  - Position ($[x, y]$)
  - more information, like path length, curvature, heading angle, etc, can be added ($[x, y, s, \kappa, \psi]$)




- **Trajectory Planning:**
  - Position and velocity ($[x, y, v]$)
  - more information, like acceleration, can be added ($[x, y, v, a]$)



# Outline

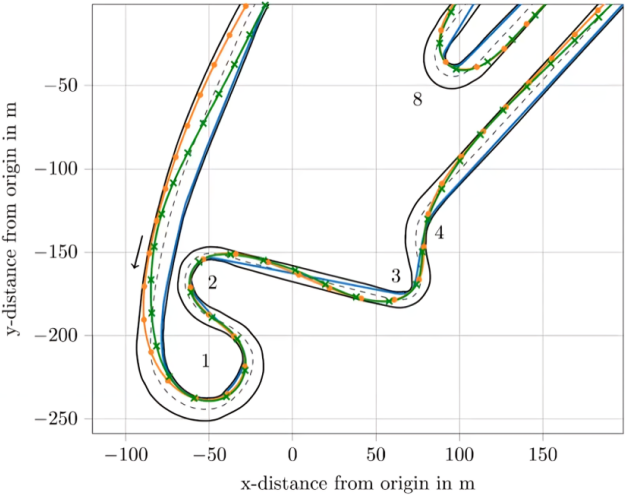
# Raceline Optimisation: Methods

**Find the optimal raceline**
Different approaches possible:

- Shortest Path (geometric QP)
- Minimum Curvature (geometric QP)
- Minimum Time (optimal control)

**Differences:**

- Solutions differ in computation runtime
- Solutions differ in complexity (in particular, if the vehicle dynamics are taken into account)
- Vehicle dynamics may not match the modelled dynamics



# Breaking Down the Problem

**Trajectory calculation by means of optimisation:**
Solving **two** subsequent sub-problems:


- Minimum distance path (Geometric QP) + Velocity Planning
- Minimum curvature path (Geometric QP) + Velocity Planning
- Minimum time (Optimal control)

Advantages of **Geometric QP** :

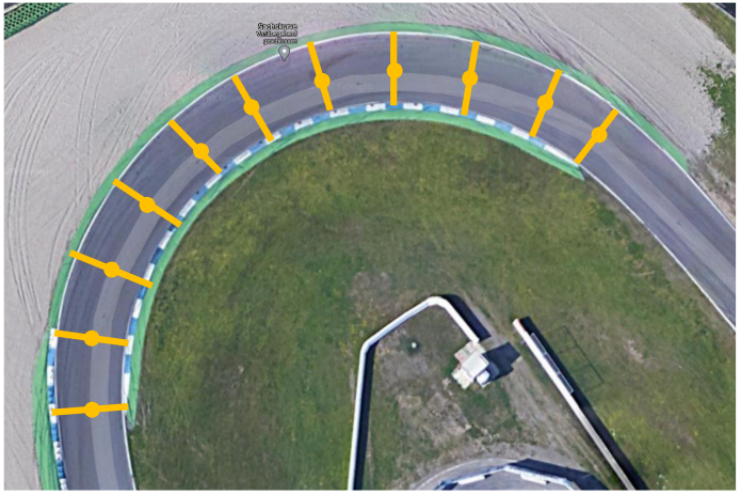
- Few parameters required
- Fast calculation times

Advantages of **optimal control**:

- Solve two sub-problems together
- Take system dynamics of vehicle into account

# Track Representation

- Representation by **centerline points and track widths**
- Discretisation distance dependent on use case (size of track)

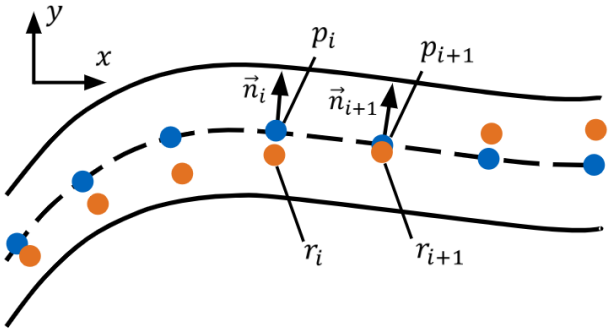


# Raceline Representation

Shifting each point along its normal vector within the track widths

$$\vec{r}_i = \vec{p}_i + \alpha_i \, \vec{n}_i$$

$$\alpha_i \in \left[ -w_{\text{tr,left,i}} + \frac{w_{veh}}{2}, w_{\text{tr,right,i}} - \frac{w_{veh}}{2} \right]$$




**Minimisation of a given objective function:**

- E.g.: Minimising the sum of the curvature values for all discretisation points
- E.g.: Minimising the sum of the distances for all discretisation points
- Consideration of the vehicle width (+ safety distance)

# Shortest Path QP

- Optimisation problem:

$$\min_{[\alpha_1 \dots \alpha_N]} \sum_{i=1}^{N} d_i^2 \quad \text{subject to} \quad \alpha_i \in [\alpha_{i,\min}, \alpha_{i,\max}] \quad \forall\, 1 \leq i \leq N$$

- Calculation of the squared distances:

$$\Delta p_{x,i} = x_{i+1} - x_i + \alpha_{i+1}\vec{n}_{i+1} - \alpha_i \vec{n}_i$$

$$d_i^2 = \Delta p_{x,i}^\top \Delta p_{x,i} + \Delta p_{y,i}^\top \Delta p_{y,i}$$

- Several matrix reshaping operations lead to an implementable formulation of the problem

---

F. Braghin, F. Cheli, S. Melzi, and S. Edoardo. "Race driver model." Computers & Structures 86, no. 13-14 (2008): 1503-1516.

# Curvature Continuous Cubic Splines

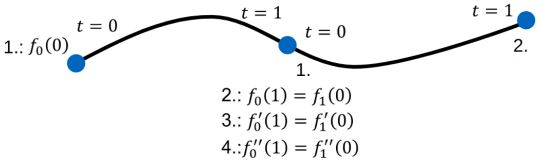
- The raceline can be represented by cubic polynomials for each segment (shown for the x-component):

$$x_i(t) = a_i + b_i t + c_i t^2 + d_i t^3$$
$$\dot{x}_i(t) = b_i + 2c_i t + 3d_i t^2$$
$$\ddot{x}_i(t) = 2c_i + 6d_i t$$

- Normalized parameter $t_i(s) = \frac{s - s_{i0}}{\Delta s_i}$ where $s$ is curvilinear distance $(0 \leq t_i \leq 1)$

**Boundary Conditions:**

1. Spline starts at current point
2. Spline ends at subsequent point
3. Heading continuity between segments
4. Curvature continuity between segments



# Minimum Curvature QP

- Optimisation problem:

$$\min_{[\alpha_1 \dots \alpha_N]} \sum_{i=1}^{N} \kappa_i^2 \quad \text{subject to} \quad \alpha_i \in [\alpha_{i,\min}, \alpha_{i,\max}] \quad \forall 1 \leq i \leq N$$

- Curvature calculation:

$$\kappa_i = \frac{x_i' y_i'' - y_i' x_i''}{(x_i'^2 + y_i'^2)^{\frac{3}{2}}}$$

- Squared curvature:

$$\kappa_i^2 = \frac{x_i'^2 y_i''^2 - 2 x_i' x_i'' y_i' y_i'' + y_i'^2 x_i''^2}{(x_i'^2 + y_i'^2)^3}$$

- Several matrix reshaping operations lead to an implementable formulation of the problem.

---

A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann. "Minimum curvature trajectory planning and control for an autonomous race car." Vehicle System Dynamics (2020).

# Velocity Profile Planning & Forward-Backward Solver

**Assumption:** Race car operates at vehicle dynamics limits

- **Acceleration Limits:**
  - Longitudinal ($a_x$): Engine (positive) and tires/brakes (negative) $a_x = \frac{F_x}{m}$
  - Lateral ($a_y$): Maximum tire forces $a_y = v^2 \cdot \kappa$

- **Forward-Backward Solver:**
  1. Initial velocity profile estimate from max lateral acceleration ($a_{y,max}$) based on $\kappa$ profile
  2. Clip velocities exceeding maximum velocity limit
  3. **Forward pass:** Apply possible positive $a_x$ (acceleration)
  4. **Backward pass:** Apply possible negative $a_x$ (braking)

In steps 3. and 4.

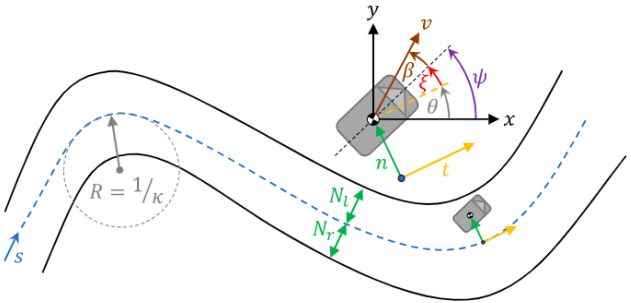
1. Calculate acting lateral acceleration: $a_{y,i} = v_i^2 \cdot \kappa_i$
2. Determine potential for longitudinal acceleration: $a_{x,i} = a_{x,max}\sqrt{1 - \left(\frac{a_{y,i}}{a_{y,max}}\right)^2}$
3. Calculate velocity in the next point, assuming: $v_{i+1} = \sqrt{v_i^2 + 2 \cdot a_{x,i} \cdot l_i}$

**We can use your method to get better velocity profiles!!**

**Minimum time trajectory planning (Optimal Control)**

Solving one single problem: simultaneous Path + Velocity Profile Planning

$$\dot{s} = \frac{v\cos(\xi+\beta)}{1-n\kappa},$$
$$\dot{n} = v\sin(\xi+\beta),$$
$$\dot{\xi} = \omega_z - \kappa\frac{v\cos(\xi+\beta)}{1-n\kappa},$$




Leveraging the **double track model** to model the dynamics of the car and obtaining a set of ODEs for $\dot{v}$, $\dot{\beta}$, and $\dot{\omega}_z$.

# Minimum Time Trajectory Planning

The formulation of the optimal control problem for the minimum lap time problem is:

$$\min_{x,u} \int_{s_0}^{s_f} L(x(s), u(s))\, ds$$

s.t.

$$\frac{dx}{ds} - f(x(s), u(s)) = 0,$$
$$g(x(s), u(s)) = 0,$$
$$h(x(s), u(s)) \leq 0,$$
$$r(x(s_0), x(s_f)) \leq 0,$$

where

$$L(x(s), u(s)) = \frac{dt}{ds} = \frac{1 - n\kappa}{v \cos(\xi + \beta)}.$$

The constraints are based on the dynamics and geometric constraints.

F. Christ, A. Wischnewski, A. Heilmeier, and B. Lohmann. "Time-optimal trajectory planning for a race car considering variable tyre-road friction coefficients." Vehicle system dynamics 59, no. 4 (2021): 588-612.

# Minimum Time-Curvature

- The raceline can be represented by cubic polynomials for each segment (shown for the x-component):

$$x_i(t) = a_i + b_i t + c_i t^2 + d_i t^3$$
$$\dot{x}_i(t) = b_i + 2c_i t + 3d_i t^2$$
$$\ddot{x}_i(t) = 2c_i + 6d_i t$$

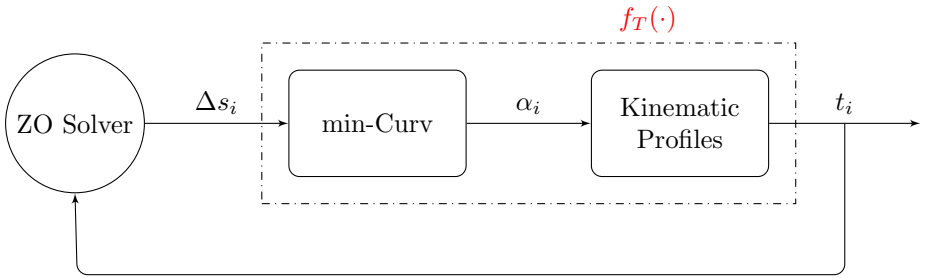
- Normalised parameter $t_i(s) = \frac{s - s_{i0}}{\Delta s_i}$ where $s$ is curvilinear distance $(0 \leq t_i \leq 1)$
- In minimum curvature method, $\Delta s_i$ is fixed at the value of $\|\vec{p}_{i+1} - \vec{p}_i\|$.
- We propose a method to use $\Delta s_i$ as a variable and try to minimise the lap time with respect to it.
- Thus, in an iterative manner, **a)** we fix $(\Delta s_i)_k$, find the path and trajectory using the minimum curvature method, **b)** find the lap time, and **c)** use the lap time as a feedback to adjust $(\Delta s_i)_{k+1}$.

**Optimisation Problem:**

$$\min_{\Delta s} f_T(\Delta s) \quad \text{subject to} \quad \Delta s_i \in [\Delta s_{i,\min}, \Delta s_{i,\max}] \quad \forall 1 \leq i \leq N-1,$$

where $f_T(\cdot)$ takes $\Delta s$ as input and its output is the lap time obtained through the minimum curvature method.

- We solve this optimisation problem using zeroth-order solvers like Gaussian random oracle based methods or evolution strategy methods (CMA-ES).

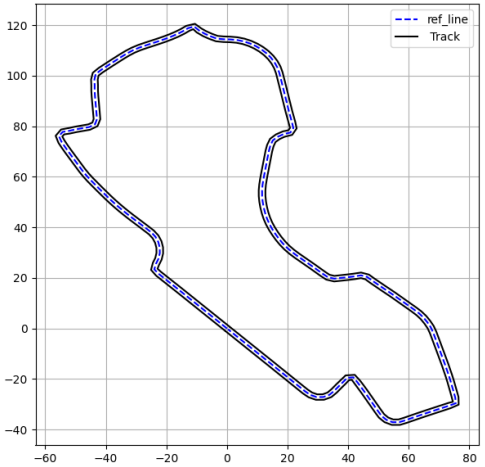


# Outline

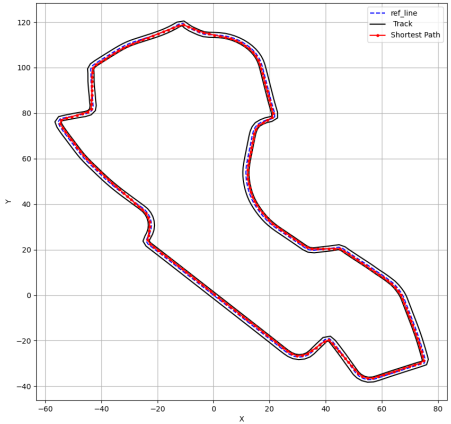
# Melbourne F1tenth Map

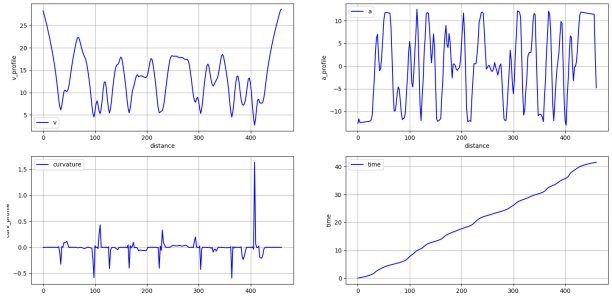
**We will implement:**

- Shortest Path with vanilla forward-backward
- Minimum Curvature with vanilla forward-backward
- Minimum time optimal control
- Min Time-Curv (Gaussian and CMA-ES) with vanilla forward-backward
- Min Time-Curv (Gaussian and CMA-ES) with clothoid forward-backward



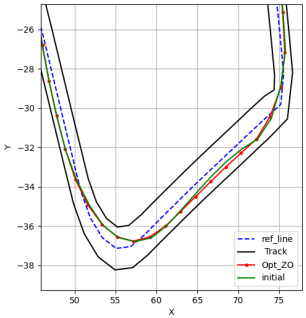
# Shortest Path with vanilla forward-backward

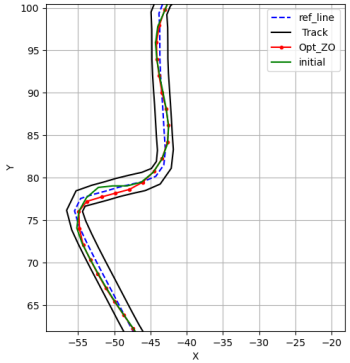


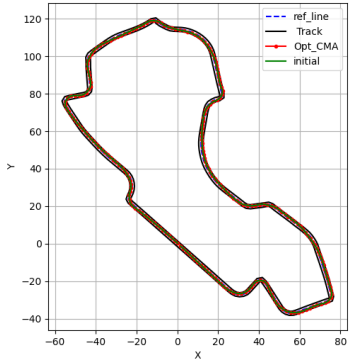




- Lap time (Shortest Path): 41.67 s

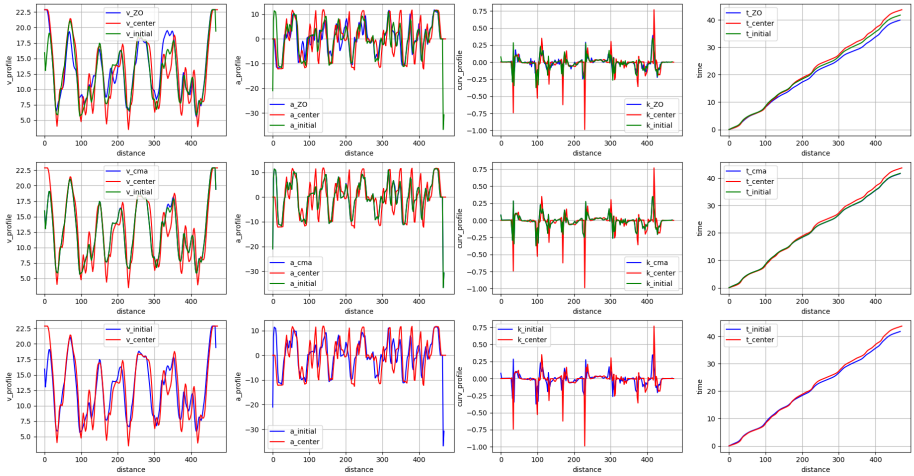
# Min Curv and Min Time-Curv with vanilla forward-backward



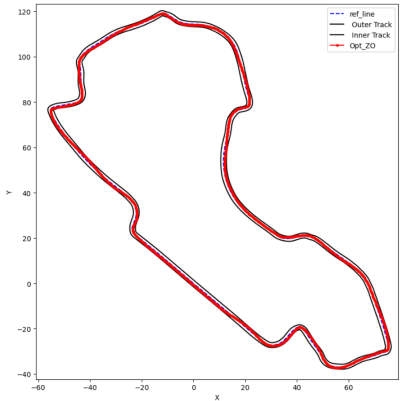



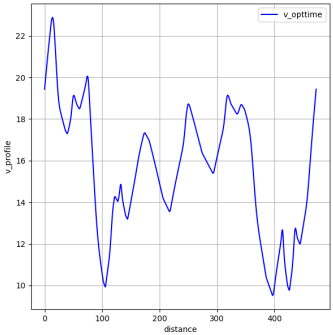
# Min Curv and Min Time-Curv with vanilla forward-backward

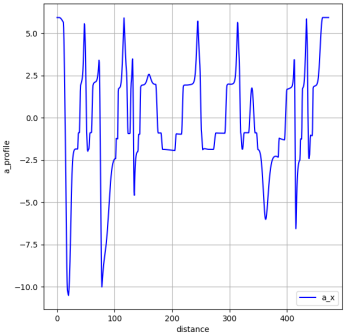



- Lap time (centre-line): 43.79 s
- Lap time (min curv): 41.85 s
- Lap time (time-curv Gaussian): 39.74 s
- Lap time (time-curv CMA-ES): 41.75 s

# Min Time optimal control

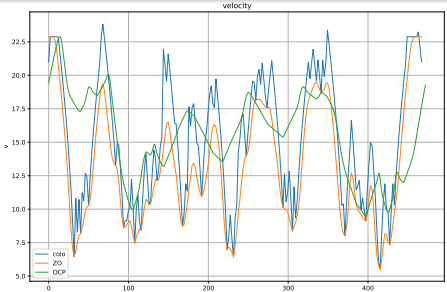


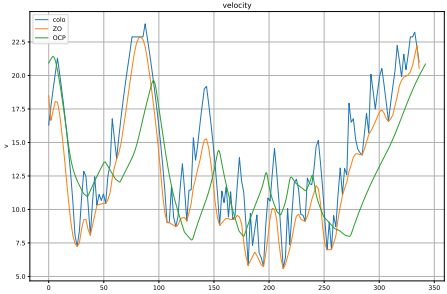




- Lap time (OCP): 31.424 s

# Min Time-Curv Clothoid forward-backward




| Method (Melbourne) | Lap Time(s) |
|---|---|
| Centerline | 43.79 |
| Shortest Path | 41.67 |
| Min Curv | 41.85 |
| Min Time | 31.43 |
| Min Time Curv (CMA-ES) | 41.75 |
| Min Time Curv (Gauss-vannila) | 39.74 |
| Min Time Curv (Gauss-clothoid) | 33.95 |

| Method (Sao Paulo) | Lap Time(s) |
|---|---|
| Centerline | 34.92 |
| Shortest Path | 32.93 |
| Min Curv | 31.61 |
| Min Time | 28.54 |
| Min Time Curv (CMA-ES) | 31.53 |
| Min Time Curv (Gauss-vannila) | 31.21 |
| Min Time Curv (Gauss-clothoid) | 26.67 |

# Outline

# OptiLine Python Package

**Provide a user-friendly Python package that takes the map's centerline and boundary details as input, and outputs an optimised path and trajectory, in a few lines of Python code.**

**Path**
- **Shortest Path**
- **Minimum Curvature**
- **Minimum Time**
- **Min Time-Curv**

**Kinematic Profiles**
- **Vanilla Forward-Backward**
- **Optimal Control (min time)**[1]
- **Clothoid path(Your Method)**[2,3]

---

1. Parts of the current code is adopted from `https://github.com/TUMFTM/global_racetrajectory_optimization`.
2. Frego, M., Bertolazzi, E., Biral, F., Fontanelli, D., & Palopoli, L. (2017). Semi-analytical minimum time solutions with velocity constraints for trajectory following of vehicles. Automatica, 86, 18-28.
3. It turned out that the results in the above paper do not capture all real-world constraints, and that's why it gives us a better answer than the optimal control method in one of the numerical examples in the previous slides. They have corrected the problem in their newer version of work.

# Future Work

## Wrapping up

- Finalise and improve the codebase for path generation.
- Finalise and improve the codebase for computing kinematic profiles.

## Future Work

- Integrate your C++ implementation into the Python package as an alternative method for velocity profile computation.
- Use the integrated method as a substitute for the feedback of the ZO solvers.
- Develop a beta version of the package, including a user instruction file, and make it publicly available.

# Brainstorm

## Discussion

- Any suggestions on the layout of the project?
- (If you are okay) Any suggestions on the way to integrate your code into the Python package?
- Any other interesting raceline optimisation method to add to the package?
- Any new idea on how to improve the project?

## AI Solver

- We can construct a dataset using $\mathcal{O}(100)$ maps along with their racelines.
- We can try to design and train a neural network using the constructed dataset.

Thanks!